دفتر 19 / من الشرح للتطبيق

نعمل آلة حاسبة صغيرة

هنجمع اللي اتعلمناه في برنامج واحد: المستخدم يختار عملية، يدخل رقمين، ويشوف النتيجة أو رسالة واضحة لو في مشكلة. الحساب في دالة والواجهة في دالة تانية.

في آخر الدفتر هتقدر تعمل إيه؟

تشغل آلة حاسبة بأربع عمليات، وتختبر القسمة على صفر والمدخل غير الرقمي، وتضيف عملية جديدة بنفس النمط.

هنمشي بالترتيب ده

1. نحدد المدخلات والنتيجة والحالات المرفوضة.
2. نفصل دالة الحساب عن القراءة والطباعة.
3. نربط الجزأين ونختبر مسارات النجاح والفشل.

قبل ما تبدأ

الدوال والشروط وfloat وtry/except. اكتب جزأي الكود في الصفحتين 2 و3 داخل ملف calculator.py واحد، بنفس الترتيب.

افتح ملف كود جنب الدفتر. اقرأ المثال، توقع النتيجة، وبعدها شغله. لو النتيجة مختلفة، ارجع للسطر اللي غيرها بدل ما تعيد نسخ الكود كله.

خريطة الدفتر: الفكرة ص 2 • تطبيق خطوة بخطوة ص 3 • الأخطاء ص 4 • تمارين ص 5 • حلول ص 6 • مراجعة ص 7

بايثون مع نوار

المشروع الأول

شرح • أمثلة • تمارين • حلول

19

01 / الجزء الأول

الحساب من غير واجهة

calculate تستقبل عددين وعملية نصية. ترجع عددا عند النجاح، وترفع خطأ لو العملية غير مدعومة أو القسمة على صفر. كده نقدر نختبر الحساب بدون كتابة إدخال يدوي كل مرة.

PYTHON

```python
def calculate(a, b, operation):
    if operation == "+":
        return a + b
    if operation == "-":
        return a - b
    if operation == "*":
        return a * b
    if operation == "/":
        if b == 0:
            raise ZeroDivisionError("Cannot divide by zero")
        return a / b
    raise ValueError("Choose +, -, * or /")
```

كل فرع يرجع فورا، لذلك لا نحتاج else بين الفروع هنا. التحقق من الصفر قبل القسمة يتيح رسالة واضحة. ولو لم يتطابق أي اختيار، آخر سطر يرفض العملية.

جرب الدالة وحدها

بعد تعريفها، استدع calculate(8, 2, "+")؛ النتيجة المتوقعة 10. غير العملية إلى / فتكون 4.0.

02 / الجزء الثاني

الإدخال والرسائل

```python
def main():
    operation = input("Operation (+ - * /): ").strip()
    try:
        a = float(input("First number: "))
        b = float(input("Second number: "))
        result = calculate(a, b, operation)
    except (ValueError, ZeroDivisionError) as error:
        print("Problem:", error)
    else:
        print(f"Result: {result:g}")

if __name__ == "__main__":
    main()
```

float يحول النص لعدد. except يلتقط النوعين المتوقعين، وas error يتيح قراءة رسالة الخطأ. else يعرض الناتج فقط عند النجاح. التنسيق g يعرض العدد بشكل مختصر بدل أصفار عشرية غير لازمة.

شرط __name__ في النهاية يشغل main لما الملف ينفذ مباشرة، ويسمح باستيراد calculate للاختبار بدون فتح أسئلة الإدخال. شغل الملف من الطرفية بالأمر التالي.

```terminal
python calculator.py
```

ماذا سيظهر؟

عند اختيار * وإدخال 6 ثم 7، تظهر Result: 42. لو كتبت نصا بدل رقم، تظهر رسالة Problem بدل توقف البرنامج برسالة traceback.

03 / نختبر المشروع

النجاح وحده مش كفاية

أربع عمليات وحالتان مرفوضتان

اختبر الجمع 8 و2 فيطلع 10، والطرح فيطلع 6، والضرب فيطلع 16، والقسمة فيطلع 4. جرب القسمة على صفر، واختيارا غير مدعوم، ونصا غير رقمي. كل حالة لها نتيجة أو رسالة محددة.

حدود النسخة التعليمية

البرنامج ينفذ عملية واحدة ثم ينتهي. لا يحلل تعبيرا طويلا مثل 2+3*4، ولا يستخدم eval. float مناسب للتجربة، لكنه ليس نموذجا للحسابات المالية الدقيقة. المدخلات المتقدمة مثل nan وinf ليست جزءا من تمرينا.

ترتيب التحقق

حاليا نقرأ العددين ثم نرفض العملية غير المدعومة داخل calculate. تحسين ممكن: افحص اختيار العملية قبل طلب الأرقام حتى لا تطلب بيانات لن تستخدمها.

اختبار مباشر للدالة

احتفظ بدالة الحساب خالية من input وprint. ده يخلي اختبارها بقيم ثابتة سهلا، ويسمح بإعادة استخدامها في واجهة أخرى. لا ترجع أحيانا رقما وأحيانا رسالة خطأ نصية من نفس المسار الحسابي.

تحدي صغير

أضف عملية الأس **. حدد سلوكها قبل التعديل، ثم اختبر 2 و3 علشان النتيجة تكون 8. لا تغير العمليات الموجودة دون داع.

جرب قبل ما تبص على الحل

دورك إنت

تطبيق 01 / اختبار مكتوب

اكتب ثلاث حالات: قسمة عادية، قسمة على صفر، ومدخل نصي غير صالح. سجل المتوقع لكل واحدة.

تلميح صغير

الخطأ قد يأتي من التحويل أو الحساب.

تطبيق 02 / عملية جديدة

أضف ** إلى calculate واختبر 2 و3.

تلميح صغير

ضع فرعا قبل raise الأخيرة، وحدث رسالة الخيارات.

تطبيق 03 / منع سؤال غير لازم

كيف تمنع طلب الأعداد عند اختيار عملية غير مدعومة؟

تلميح صغير

افحص operation في main قبل try الخاصة بالأعداد.

قبل صفحة الحل: جرب حالة عادية وحالة على الحدود. اكتب النتيجة المتوقعة الأول، وبعدها قارنها باللي طلع فعلا.

مساحة لتجربتك

حلول مشروحة / قارن طريقة التفكير

نراجع اللي عملناه

حل 01 / اختبار مكتوب

8 / 2 تعطي 4. القسمة على صفر تعرض المشكلة. إدخال hello يسبب ValueError أثناء float ويظهر كرسالة Problem.

حل 02 / عملية جديدة

```python
if operation == "**":
    return a ** b
```

أضف الفرع التالي داخل الدالة، قبل رفض العملية. الناتج 8.

حل 03 / منع سؤال غير لازم

```python
if operation not in ("+", "-", "*", "/"):
    print("Unsupported operation")
    return
```

أضف الفحص التالي بعد قراءة العملية. return تنهي main.

ورقة ترجع لها بسرعة

قبل ما تقفل الدفتر

راجع نفسك

- جمعت الجزأين في ملف واحد.
- فصلت الحساب عن واجهة الإدخال.
- اختبرت النجاح والأخطاء والاختيارات غير المدعومة.

المصطلحات من غير تعقيد

Validation : التحقق من صلاحية المدخلات.

Interface : الجزء الذي يقرأ من المستخدم ويعرض له.

Test case : مدخلات محددة ومعها نتيجة متوقعة.

اختبار الفهم

لو calculate ترجع رسالة نصية بدل رفع الخطأ، قد يحاول كود آخر جمعها مع عدد. الحفاظ على عقد واضح للدالة يمنع هذا الخلط.

نكمل على إيه؟

الدفتر 20: لعبة تخمين تستخدم حالة متغيرة ومحاولات وإعادة إدخال.

لما تحتاج التفاصيل الرسمية

الشرح والأمثلة هنا مكتوبة للمبتدئ. الروابط التالية مراجع للمراجعة والتوسع، وليست نصا منقولا منها.

الدوال والشروط

https://docs.python.org/3/tutorial/controlflow.html

الأخطاء

https://docs.python.org/3/tutorial/errors.html